Hardware Architecture for Positive Definite Matrix Inversion Based on LDL Decomposition and Back-Substitution

Carl Ingemarsson and Oscar Gustafsson
Department of Electrical Engineering, Linköping University
SE-581 83 Linköping, Sweden
E-mail: {carl.ingemarsson, oscar.gustafsson}@liu.se

***Abstract*—In this paper we propose an efficient hardware architecture for computation of matrix inversion of positive definite matrices. The algorithm chosen is LDL decomposition followed directly by equation system solving using back substitution. The architecture combines a high throughput with an efficient utilization of its hardware units. We also report FPGA implementation results that show that the architecture is well tailored for implementation in real-time applications.**

I. Introduction

Inversion of positive definite matrices is required in many different application areas, e.g. MIMO communications [1], adaptive filtering and image analysis. There are a few different algorithms available that can utilize the structure of a positive definite matrix when computing its inverse. If an algorithm for general matrix inversion is used and, hence, this structure is not utilized, as in e.g. [2], a lot of unnecessary operations are computed resulting in a much less efficient solution.

In [3], it was shown that an algorithm combining LDL decomposition and direct back-substitution [4] is better suited for hardware implementation than the perhaps more straightforward algorithm based on LDL decomposition, triangular inversion and matrix multiplication. This matrix multiplication based inversion algorithm was used with Cholesky decomposition in e.g. [5], [6]. Furthermore it was shown in [3] that block-wise matrix inversion [7]–[10] needs much larger word lengths to achieve similar precision at a similar number of operations. Cholesky decomposition can also be combined with back-substitution, but LDL factorization uses less multiplications and does not require computation of (inverse) square roots [3].

We have not found any earlier implementation of this algorithm. LDL decomposition and equation system solving is used in [11] and ASIC implementation results are reported, but there the positive definite matrix inverse is not explicitly computed and hence any direct comparison is possible. Systolic architectures for computation of LDL decomposition alone was proposed in [12], [13].

Hardware implementation of matrix inversion has been addressed by several authors e.g. in [2], [5]–[9], [14]–[18]. In most cases algorithms for general matrix inversion was used. However in [5], [9], [15] inversion of positive definite matrices was considered.

The organization of the rest of this paper is as follows: In Section II the chosen algorithm is described. Then, in Section III the proposed architecture is presented. FPGA implementation results are reported in Section IV. In Section V the implementation results are compared with earlier published results. Finally, Section VI concludes the paper.

II. Algorithm

The algorithm we have chosen for implementation can be described as a combination of LDL factorization and back substitution. In the following we will however consider the upper-diagonal equivalent $R^H D R$ factorization. This algorithm requires few multiplications. If N is the order of the matrix to be inverted only $\frac{1}{2}N^3+\frac{1}{2}N^2-N$ multiplications are needed. Furthermore it requires no square roots, and only N reciprocal operations. This algorithm is know since several decades and was e.g. in [4] presented as one candidate algorithm for positive definite matrix inversion. In [4] it is however formulated in a different manner and instead referred to as the *Bordering algorithm*.

Let us now briefly describe this algorithm. Let $\mathbf{A}$ be the matrix to be inverted and consider the system of equations $\mathbf{RX}=\mathbf{B}$, where $\mathbf{X}=\mathbf{A}^{-1}$. Then:

$$\mathbf{RX}=\mathbf{RA}^{-1}=\mathbf{R}\left(\mathbf{R}^H\mathbf{DR}\right)^{-1}=\mathbf{D}^{-1}\left(\mathbf{R}^H\right)^{-1}.$$

Hence $\mathbf{B}=\mathbf{D}^{-1}\left(\mathbf{R}^H\right)^{-1}$ and then $\mathbf{B}$ is zero above its diagonal and (all but one of) its diagonal entries have

978-1-5386-3954-2/16/$31.00 ©2016 IEEE

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on September 17,2026 at 17:55:20 UTC from IEEE Xplore. Restrictions apply.

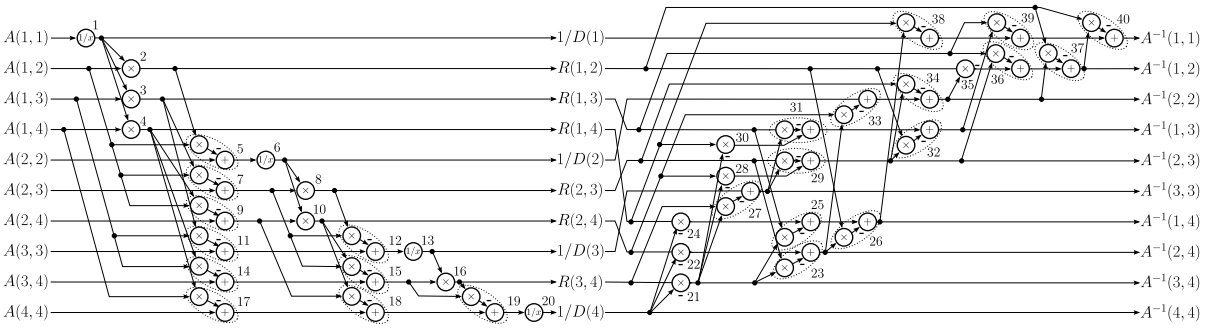


Fig. 1. Signal flow graph of the algorithm for inversion of a 4×4 matrix.

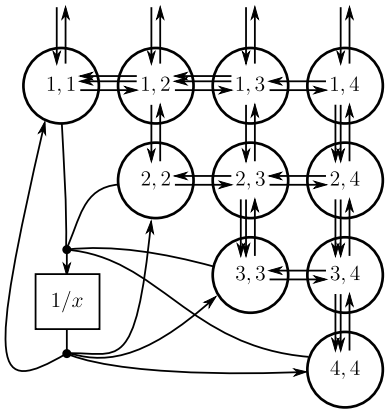


Fig. 2. Architecture for inversion of 4×4 matrices.

already been computed in the preceding factorization. There is therefore no need of computing the right hand side of the system of equations if it is solved using its upper diagonal part. The solution can therefore directly be computed using back-substitution.

In Fig. 1 a signal flow graph of this algorithm for the case of a 4×4 inversion is depicted. The algorithm is equally applicable to real or complex matrices. For the complex case some complex conjugations are missing in Fig. 1.

III. Proposed Architecture

Our proposed architecture consists of a triangular array of $\frac{N(N-1)}{2}$ cells. For the case of 4×4 the number of cells is ten. Such an array is depicted in Fig. 2. Each cell consists of a Multiply-and-Accumulate (MAC) unit and a set of small memories for keeping intermediate computation results and data to be communicated to other cells. The input and output data are communicated through the top row of cells. An efficient mapping of the operations seen in the signal flow graph is achieved if operations involved in the computation for elements in matrix $\mathbf{R}$ $(1/\mathbf{D})$ and $\mathbf{A}^{-1}$ are computed in the cell with the same index, e.g operation 4, 24, 25, and 26 in the top right cell. With this mapping each cell is assigned four operations. In case of inner cells this is four MAC-operations. For diagonal cells it is one (real) reciprocal operations and three complex MAC operations each resulting in a real result. This means that these cells each only needs half of one ordinary complex MAC unit. With a proper scheduling it is then possible to achieve almost 100% utilization of the hardware units. If the reciprocal operations are also properly scheduled only one single reciprocal unit can be shared for computations of all reciprocal operations. Then the utilization of the reciprocal unit will be 100% and of the MAC units in the inner and diagonal cells 100% and $\frac{N-1}{N}$, respectively.

As can be seen in Fig. 2 the cells can communicate north-south or east-west. Different number of buses are needed depending on the number of data variables that need to be communicated. Not more than two in one direction and one in the other are needed in any case.

The scheduling has been performed using an optimization technique. The optimization problem was formulated such that the latency of the MAC and reciprocal processing elements can be set individually. Further the communication between neighboring cells was allocated one clock cycle. The scheduling was constrained such that it should be possible to start one new computation every N clock cycles. This means that the scheduling of operations is performed over N clock cycles. In each time unit processing corresponding to several different input matrices will therefore take place. How many depends on the total latency of the scheduled algorithm which in turn depend on the latency of the processing elements. As goal function for the optimization problem

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on September 17,2026 at 17:55:20 UTC from IEEE Xplore. Restrictions apply.

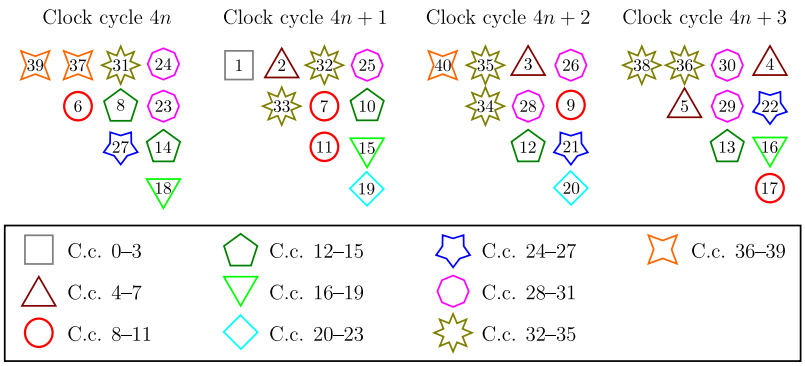


Fig. 3. Scheduling of the operations in Fig. 1 on the architecure in Fig. 2.

minimization of the total life time of all memory variables was chosen. The input values are sent into the array during the first N clock cycles. The outputs variables are all used for computation in the first row and are sent to the outputs when available there, subject to the scheduling modulo N. For the case of 4×4 and choosing the latency of the MAC units and reciprocal as one and three respectively this is between clock cycle number 28 and 39. This schedule is shown in Fig. 3. The numbering is defined by the numbers attached to operations in the signal flow graph in Fig. 1.

IV. FPGA IMPLEMENTATION

We have implemented the architecture both for the case of complex 4×4 and 8×8 positive definite matrices. In both cases we have set the latency of MAC units as one and of the reciprocal unit as three clock cycles. In the design a word length of 18 bits per real dimension has been used, which is a suitable choice given the FPGA architecture. Out of these 18 bits, based on numerical simulations, six bits are used as integer bits and twelve as fractional.

The complex MAC units where implemented using four real multiplications each except for the diagonal cells, where only two real multiplications are needed as discussed above. At least for the inner cells it would be possible to lower the number of DSP48E1 blocks further by implementing the complex multipliers with three real multipliers instead of four. One advantage of this approach would be that the pre-adders in the DSP blocks can be utilized [19]. This would however also result in some additional extra slice resources and possibly increase the critical path of the design with lower f_{mac} as a result. We have chosen not to explore this trade-off further our implementation.

We have targeted the FPGA part *xc7vx330t-3-ffg1157* and used Xilinx ISE tool of version 14.7.

A. Reciprocal Unit

The implementation of the reciprocal unit is based on the Newton-Raphson algorithm. At the input the data is normalized and converted to unsigned representation. A number of more significant bits are then used to compute an estimate via a look-up-table (LUT). The estimate is then fed to a unit that computes one Newton-Raphson iteration. This requires two multiply operations. At the output the data is de-normalized and negated if the input value was negative. The entire reciprocal unit is organized as a pipeline that can start a new reciprocal computation in each clock cycle. For the implementation results reported below the LUT in the reciprocal unit have 8 input bits and 9 output bits.

V. RESULTS

Table I presents implementation results of the proposed design and also some earlier published results for inversion of 4×4 matrices. As can be seen the proposed design uses 34 DSP48E1 blocks. Four blocks are used for each MAC unit in the inner cells, two for each MAC unit in the diagonal cells and two for the reciprocal unit. The low multiplication count of the algorithm combined with the high utilization of DSP blocks yields a high throughput per DSP48E1. A significant number of the FPGA LUTs and registers used by our proposed design are for implementing memories in the cells.

In [2] FPGA implementation results of QR decomposition based matrix inversion is presented. This algorithm can invert general matrices. However, for the MMSE MIMO detection problem described as target application in [2] the matrix to be inverted is positive definite. The reported design does not use the DSP48 blocks

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on September 17,2026 at 17:55:20 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON WITH PUBLISHED RESULTS ON 4×4 MATRIX INVERSION ON FPGA.

	[20]	[2]	[6]	Proposed
Device	xc7vx1140t	xc4vlx200	xc4vsx35	xc7vx330t-3
Word length	20	16, 18	20	18
Data format	floating	fixed	fixed	fixed
# of slices needed	1095	53598	3682	1298
# of LUTs	1784	N/A	N/A	4231
# of Registers	2465	N/A	N/A	4454
# of block RAMs	1	N/A	1	0
# of DSP48(E1s)	9	0	12	34
Latency, # clks	52	200	N/A	40
Latency, ns @ $f_{\max}$	172	1420	N/A	250
$f_{\max}$, MHz	302	140.9	N/A	160
Throughput, MInv/s	5.80	17.62	0.25	40
TP/slice, kInv/s/#	5.29	0.33	0.068	30.8
TP/DSP48, kInv/s/#	644	N/A	20.83	1176

available in the Virtex-4 FPGA used. However, the authors of [2] estimate that using DSP blocks would reduce the number of slices by 19000. A high throughput is achieved however at a vast number of slices used. Even if Virtex-4 slices are different than Virtex-7 slices, this number would not be orders of magnitudes lower for an equivalent Virtex-7 design.

Implementation results for FPGA implementation of an architecture for approximate inversion of positive definite matrices was presented in the context of MIMO detection in [20]. A Virtex-7 FPGA was targeted also for their design. The approximation uses a Neumann series expansion with $K = 2$ terms to approximate the matrix inverse. In [21], $K = 3$ is instead used because an approximation with $K = 2$ gives too crude results.

Given that our design computes the exact inverse and that the design from [20] only computes an approximation one may expect the hardware efficiency measured as Throughput/slice and Throughput/DSP48E1 in Table I to be lower for our design. That is however not the case. We also note that our design delivers a significantly higher throughput.

In [6] implementation results for matrix multiplication based inversion using Cholesky decomposition was reported. Even though different FPGA families was targeted it is clear that our design shows better results.

In Table II implementation results for 8×8 matrix inversion are presented. Our design show significantly higher throughput and also higher throughput per slice as compared with [20]. Also here this is surprising since in [20] results for approximate inversion was reported.

TABLE II
COMPARISON WITH PUBLISHED RESULTS ON 8×8 MATRIX INVERSION ON FPGA.

	[20]	Proposed
Device	xc7vx1140t	xc7vx330t-3
# of slices needed	3985	6538
# of LUTs	9476	22383
# of Registers	8094	18138
# of block RAMs	1	0
# of DSP48(E1s)	21	131
Latency, # clks	55	88
Latency, ns @ $f_{\max}$	192	657
$f_{\max}$, MHz	285	134
Throughput, MInv/s	5.18	16.7
TP/slice, kInv/s/#	1.30	2.56
TP/DSP48, kInv/s/#	247	128

VI. CONCLUSION

In this paper we have proposed an efficient architecture for computation of positive definite matrix inversion. By interleaving the computations of several inverses a high utilization of the hardware blocks are achieved. We have also presented FPGA implementation results that compare favorable with published results.

REFERENCES

[1] M. Čirkić and E. G. Larsson, "SUMIS: Near-optimal soft-in soft-out MIMO detection with low and fixed complexity," *IEEE Trans. Signal Process.*, vol. 62, no. 12, pp. 3084–3097, Jun. 2014.

[2] L. Ma, K. Dickson, J. McAllister, and J. McCanny, "QR decomposition-based matrix inversion for high performance embedded MIMO receivers," *IEEE Trans. Signal Process.*, vol. 59, no. 4, pp. 1858–1867, Apr. 2011.

[3] C. Ingemarsson and O. Gustafsson, "On fixed-point implementation of symmetric matrix inversion," in *Proc. Europ. Conf. Circuit Theory Design*, Aug. 2015, pp. 1–4.

[4] R. J. Clasen, "Numerical methods for inverting positive definite matrices," Rand Corp., Santa Monica, CA, Research Memorandum RM-4952-PR, Aug. 1966.

[5] A. Burian, J. Takala, and M. Ylinen, "A fixed-point implementation of matrix inversion using cholesky decomposition," in *Proc. 46th Midwest Symp. Circuits and Systems*, vol. 3, Dec. 2003, pp. 1431–1434.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on September 17,2026 at 17:55:20 UTC from IEEE Xplore. Restrictions apply.

[6] A. Irturk, B. Benson, S. Mirzaei, and R. Kastner, "GUSTO: An automatic generation and optimization tool for matrix inversion architectures," *ACM Trans. Embed. Comput. Syst.*, vol. 9, no. 4, pp. 321–3221, Apr. 2010.
[7] J. Eilert, D. Wu, and D. Liu, "Efficient complex matrix inversion for MIMO software defined radio," in *Proc. IEEE Int. Symp. Circuits Syst.*, May 2007, pp. 2610–2613.
[8] D. Wu, J. Eilert, D. Liu, D. Wang, N. Al-Dhahir, and H. Minn, "Fast complex valued matrix inversion for multi-user STBC-MIMO decoding," in *Proc. IEEE Int. Symp. VLSI.*, 2007, pp. 325–330.
[9] S. Eberli, D. Cescato, and W. Fichtner, "Divide-and-conquer matrix inversion for linear MMSE detection in SDR MIMO receivers," in *Proc. Norchip.*, Nov. 2008, pp. 162–167.
[10] S. Yoshizawa, Y. Yamauchi, and Y. Miyanaga, "A complete pipelined MMSE detection architecture in a 4x4 MIMO-OFDM receiver," in *Proc. IEEE Int. Symp. Circuits Syst.*, May 2008, pp. 2486–2489.
[11] D. Auras, R. Leupers, and G. Ascheid, "Efficient VLSI architectures for matrix inversion in soft-input soft-output MMSE MIMO detectors," in *Proc. IEEE Int. Symp. Circuits Syst.*, Jun. 2014, pp. 1018–1021.
[12] S. J. Bellis, W. P. Marnane, and P. J. Fish, "Alternative systolic array for non-square-root cholesky decomposition," *IEE Proc. Computers and Digital Techniques*, vol. 144, no. 2, pp. 57–64, Mar. 1997.
[13] R. P. Brent and F. T. Luk, "Computing the Cholesky factorisation using a systolic architecture," in *Proc. 6th Australian Comput. Science Conf.*, vol. 5, 1983, pp. 295–302.
[14] A. El-Amawy, "A systolic architecture for fast dense matrix inversion," *IEEE Trans. Comput.*, vol. 38, no. 3, pp. 449–455, Mar. 1989.
[15] P. Salmela, A. Happonen, A. Burian, and J. Takala, "Several approaches to fixed-point implementation of matrix inversion," in *Proc. IEEE Int. Symp. Circuits Syst.*, vol. 2, Jul. 2005, pp. 497–500.
[16] M. Karkooti, J. R. Cavallaro, and C. Dick, "FPGA implementation of matrix inversion using QRD-RLS algorithm," in *Proc. Asilomar Conf. Signals Syst. Comput.*, Nov. 2005, pp. 1625–1629.
[17] F. Edman and V. Öwall, "Compact matrix inversion architecture using a single processing element," in *Proc. IEEE Int. Conf. Electron. Circuits Syst.*, Dec. 2005, pp. 1–4.
[18] C. K. Singh, S. H. Prasad, and P. T. Balsara, "VLSI architecture for matrix inversion using modified Gram-Schmidt based QR decomposition," in *Proc. Int. VLSI Design*, 2007, pp. 836–841.
[19] C. Ingemarsson, P. Källström, and O. Gustafsson, "Using DSP block pre-adders in pipeline SDF FFT implementations in contemporary FPGAs," in *Proc. Int. Conf. Field-Programmable Logic Applicat.*, Aug. 2012, pp. 71–74.
[20] M. Wu, B. Yin, A. Vosoughi, C. Studer, J. R. Cavallaro, and C. Dick, "Approximate matrix inversion for high-throughput data detection in the large-scale MIMO uplink," in *Proc. IEEE Int. Symp. Circuits Syst.*, May 2013, pp. 2155–2158.
[21] M. Wu, B. Yin, G. Wang, C. Dick, J. R. Cavallaro, and C. Studer, "Large-scale MIMO detection for 3GPP LTE: Algorithms and FPGA implementations," *IEEE J. Selected Topics Signal Process.*, vol. 8, no. 5, pp. 916–929, Oct. 2014.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on September 17,2026 at 17:55:20 UTC from IEEE Xplore. Restrictions apply.